用Verilog描述触发器与锁存器

- **电平敏感和边沿敏感的表述方法**

  例：always @(clk or d) 或always @(clk, d)

  只要en或d中任意一个信号的电平发生变化，块内语句就执行一遍。

  always @(posedge clk)

  只要clk的上升沿到来，块内语句就执行一遍。

- **边沿敏感型信号表述关键词**

  posedge（上升沿）、negedge（下降沿）

- 一个always过程语句中只能放置一种类型的敏感信号, 不能混用。

  例：always @(posedge clk or d) 是错误的

赋值语句

- **阻塞式赋值，符号“=”**
  - 一执行完当前赋值语句，目标变量立刻获得计算值。
  - 有多条阻塞式赋值语句时，前一条语句未完成赋值前，后面的语句不能被执行。
- **非阻塞式赋值，符号“<=”**
  - 有多条非阻塞式赋值语句时，右边表达式运算结束后才同时并行完成赋值操作。
- 组合逻辑建模时，采用阻塞赋值。
- 时序逻辑建模时，采用非阻塞赋值。

触发器与锁存器的Verilog HDL描述

例：D触发器

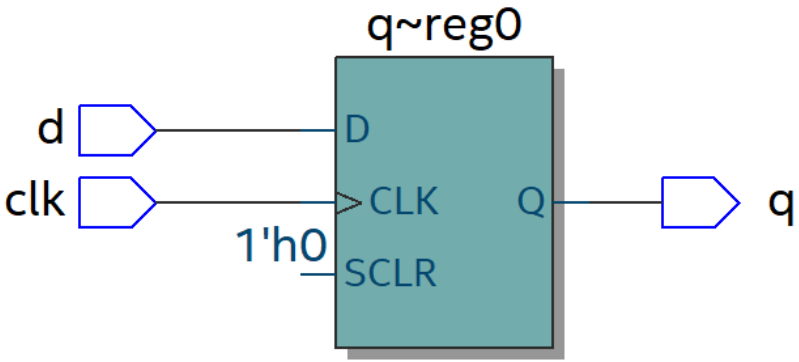
```
module dff(clk,d,q);
   input clk,d;
   output reg q;
  always @ (posedge clk)
      q<=d;
endmodule
```

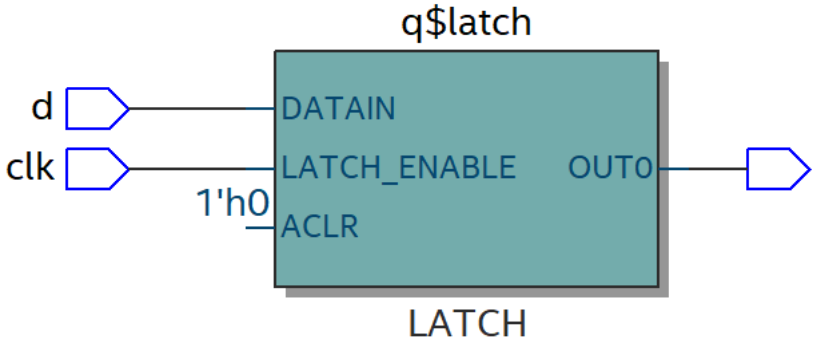
边沿敏感

D锁存器

```
module d_latch(
   input clk,d,
   output reg q);
   always @ (clk, d)
      if(clk) q<=d;
endmodule
```

电平敏感

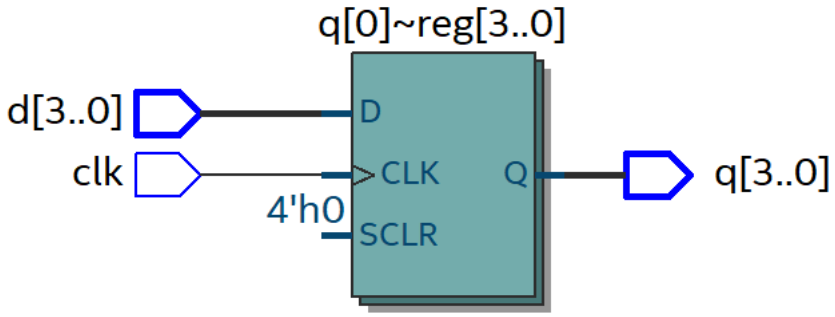




综合后的电路图

寄存器描述

```
module flop(
   input clk,
         input [3:0] d,
         output reg[3:0] q);
  always @ (posedge clk)
     q<=d;
endmodule
```



可复位寄存器描述

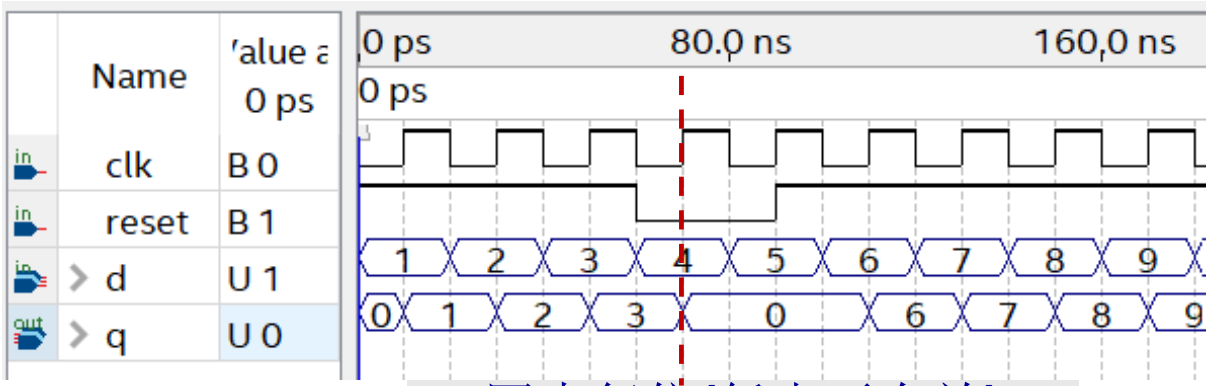
```
module flopr(
   input         clk,reset,
   input        [3:0] d,
  output reg [3:0] q);
  always @ (posedge clk)
      if (~reset) q<=4'b0;
      else        q<=d;
endmodule
```

同步复位，
低电平有效

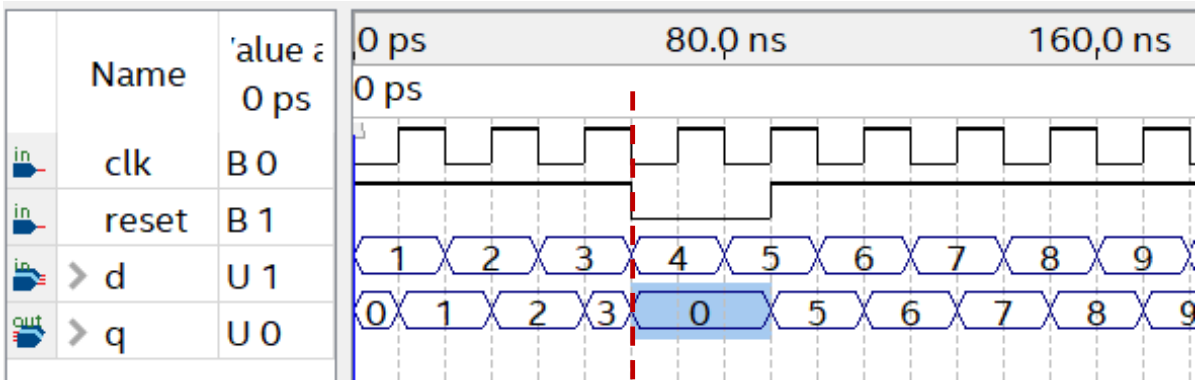
```
module flopr(
   input         clk,reset,
   input        [3:0] d,
  output reg [3:0] q);
  always @ (posedge clk ,
                  negedge reset)
      if (~rst) q<=4'b0;
      else        q<=d;
endmodule
```

异步复位
低电平有效

同步复位与异步复位仿真结果比较



同步复位(低电平有效)



异步复位(低电平有效)

阻塞/非阻塞赋值练习

分析下面2段代码，分别画出综合后的电路图

```
module test1(
   input clk,d,
   output reg q0,q1,q2);

   always @(posedge clk)
      begin
         q2=q1;
         q1=q0;
         q0=d;
      end
endmodule
```

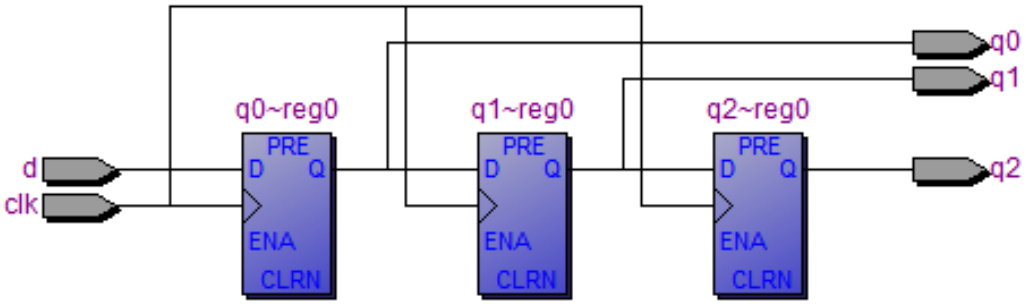
```
module test2(
   input clk,d,
   output reg q0,q1,q2);

   always @(posedge clk)
      begin
         q0=d;
         q1=q0;
         q2=q1;
      end
endmodule
```

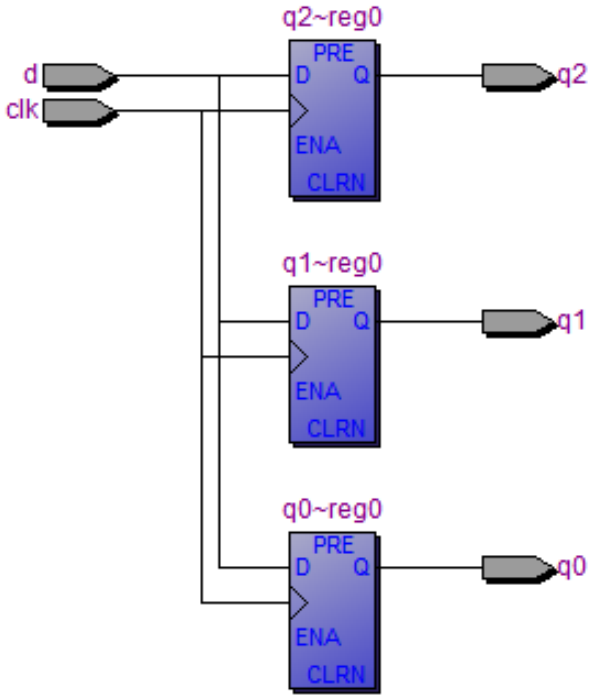
练 习

综合后的电路图

test1



test2



状态机的verilog建模

- 有限状态机（FSM）

  状态机的基本要素与分类

  状态机的基本描述方式

- 如何写好状态机

  什么是好的RTL级FSM描述

  RTL级状态机描述常用语法

  推荐的状态机描述方法

  状态机设计的其他技巧

  状态机的置位与复位

状态机概念

状态机是一种电路描述工具，它更是一种思想方法，而且状态机的HDL语言表达方式比较规范，有章可循，所以很多有经验的设计者习惯用状态机思想进行逻辑设计，对各种复杂设计都套用状态机的设计理念，从而提高设计的效率和稳定性。

状态机应用思路

- 第一种思路，从状态变量入手。如果一个电路具有时序规律或者逻辑顺序，就可以规划出状态，从这些状态入手，分析每个状态的输入，状态转移和输出，从而完成电路功能。
- 第二种思路是首先明确电路的输出关系，这些输出相当于状态的输出，回溯规划每个状态和状态转移条件与状态输入。
- 无论哪种思路，使用状态机的目的都是要控制某部分电路，完成某种具有逻辑顺序或时序规律的电路设计。

如何写好状态机

RTL级FSM的评判标准很多，最重要的几个标准如下：

- FSM要安全，稳定性高
- FSM安全是指FSM不会进入死循环，特别是不会进入非预知的状态，而且由于某些扰动进入非设计状态，也能很快地恢复到正常的状态循环中来。
- FSM设计要清晰易懂、易维护。不规范的FSM写法很难让其他人解读，甚至过一段时间后设计者也发现很难维护。
- FSM速度快，满足设计的频率要求。
- FSM面积小，满足设计的面积要求.

状态机的基本要素与分类

- 状态机的基本要素有3个：状态、输出和输入。

状态：也叫状态变量。在逻辑设计中，使用状态划分逻辑顺序和时序规律。

输出：输出指在某一个状态时特定发生的事件。

输入：指状态机中进入某个状态的条件，有的状态机没有输入条件，状态转移较为简单；有的状态机有输入条件，当某个输入条件存在时才能转移到相应的状态。

- 根据状态机的输出是否与输入条件相关，可将状态机分为两大类：摩尔(Moore)型状态机和米里(Mealy)型状态机。

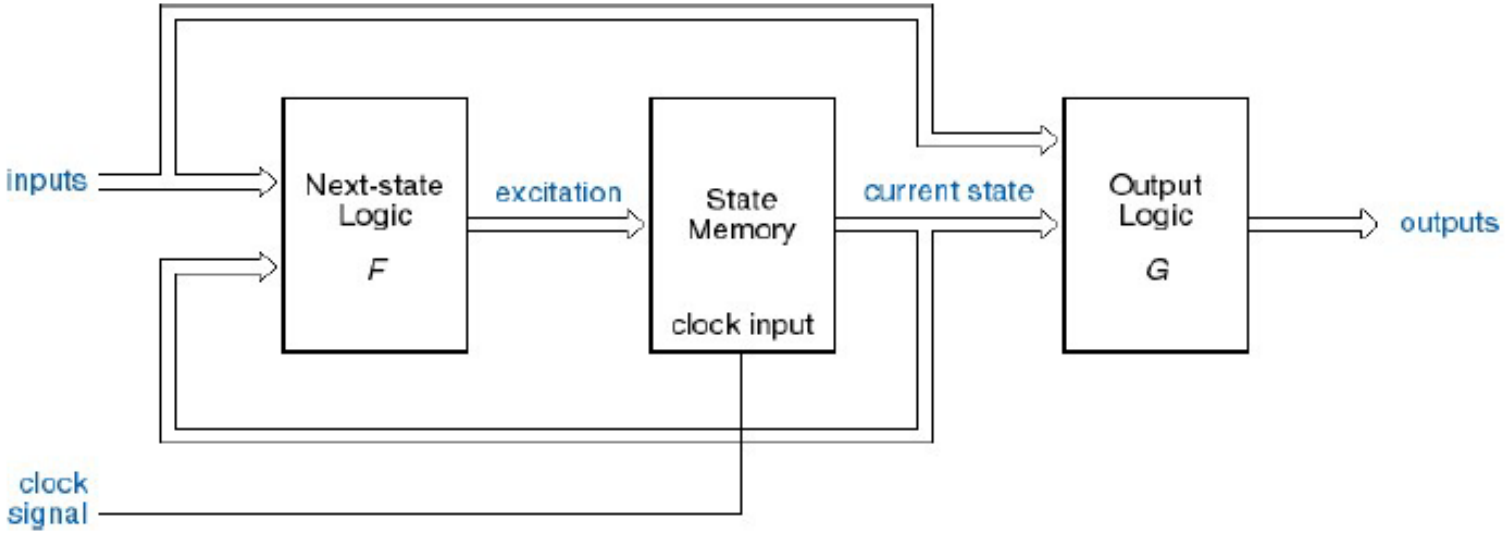
状态机的结构

状态机的结构通常包括次态逻辑、状态转移和输出逻辑三部分：

次态逻辑：根据外部输入(input)以及当前状态(current_state)确定下一状态(next_state)（组合逻辑）

状态转换：在时钟有效边沿到来时进行状态转移（同步时序）

输出逻辑：根据外部输入(input)以及当前状态(current_state) 确定当前的输出信号（组合逻辑）



RTL级状态机描述常用语法

- wire、reg：注意状态编码时一般都要使用reg寄存器型向量。
- parameter：用于描述状态编码，增强源代码可读性，简化描述。
- always：在FSM设计中有3种always的使用方法：

  第1种用法是根据主时钟沿，完成同步时序的状态转换。

  第2种用法是根据信号敏感表，完成次态逻辑的输出。

  第3种用法是根据时钟沿，完成同步时序逻辑的输出。
- case/endcase： case/endcase是FSM描述中最重要的语法关键字。为了提高设计的安全性，排除所设计的FSM进入死循环，一般要求加上default关键字来描述FSM所需状态的补集状态下的操作。

状态机描述方法（代码风格）

一段式：把状态转移、次态逻辑和输出逻辑都放在一个always过程描述（不符合将组合逻辑与时序逻辑分开描述的代码风格，代码不清晰不利于维护，不利于附加约束，不利于综合器的布局布线和设计优化，不推荐使用。

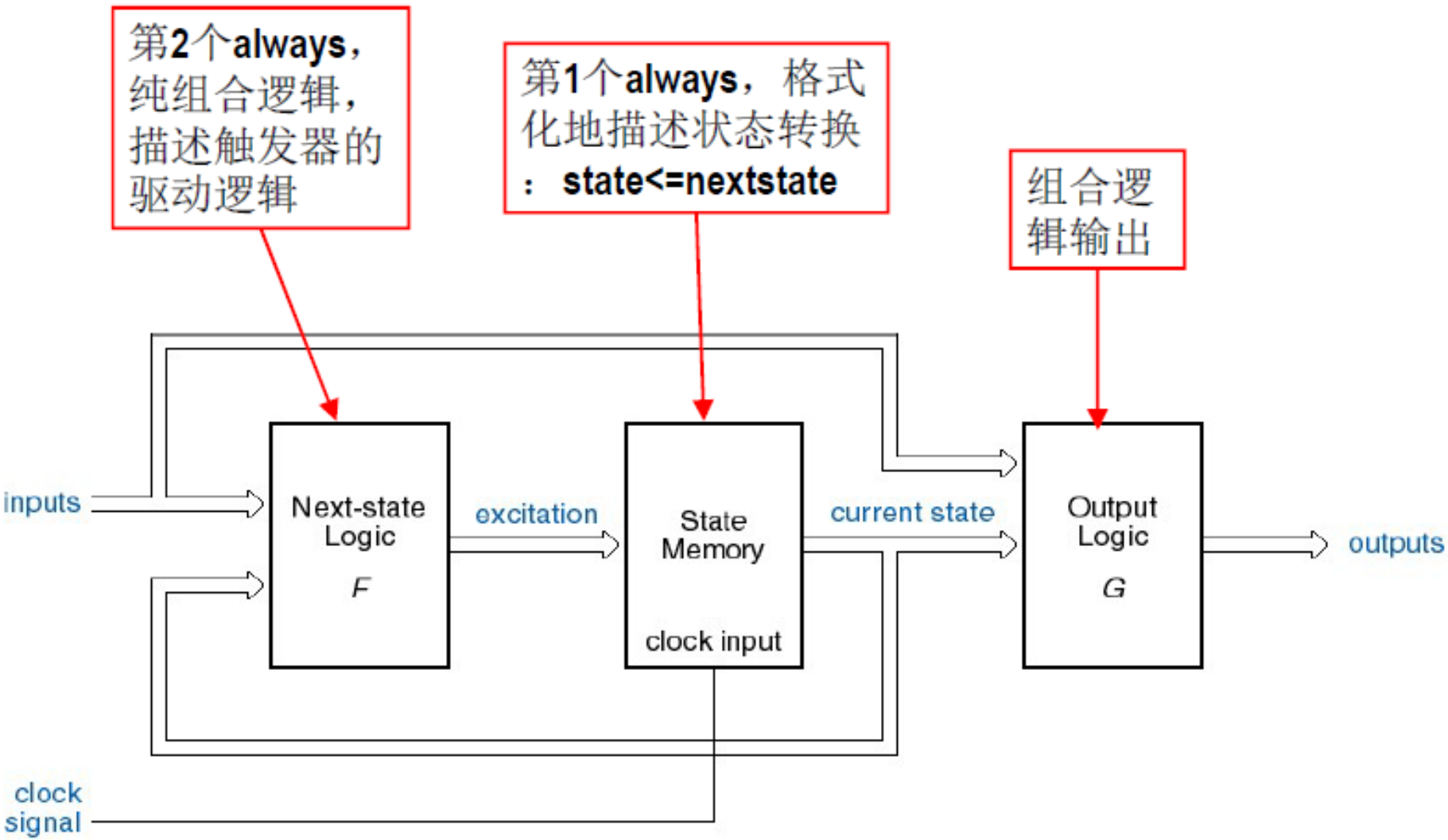
状态机描述方法（代码风格）

两段式（推荐），使用2个always模块：

- 一个always模块采用同步时序描述状态转移；
- 另一个always模块采用组合逻辑判断状态转移条件、描述状态转移规律。
- 输出逻辑可以放在第二个always 过程里面，也可单独采用组合逻辑赋值。

同步时序模块中推荐使用非阻塞赋值“<=”，
组合模块中所有赋值推荐使用阻塞赋值“=”

两段式FSM描述结构


第2个always，纯组合逻辑，描述触发器的驱动逻辑
第1个always，格式化地描述状态转换：state<=nextstate
组合逻辑输出
inputs
Next-state Logic
F
excitation
State Memory
clock input
current state
Output Logic
G
outputs
clock signal

状态机描述方法（代码风格）

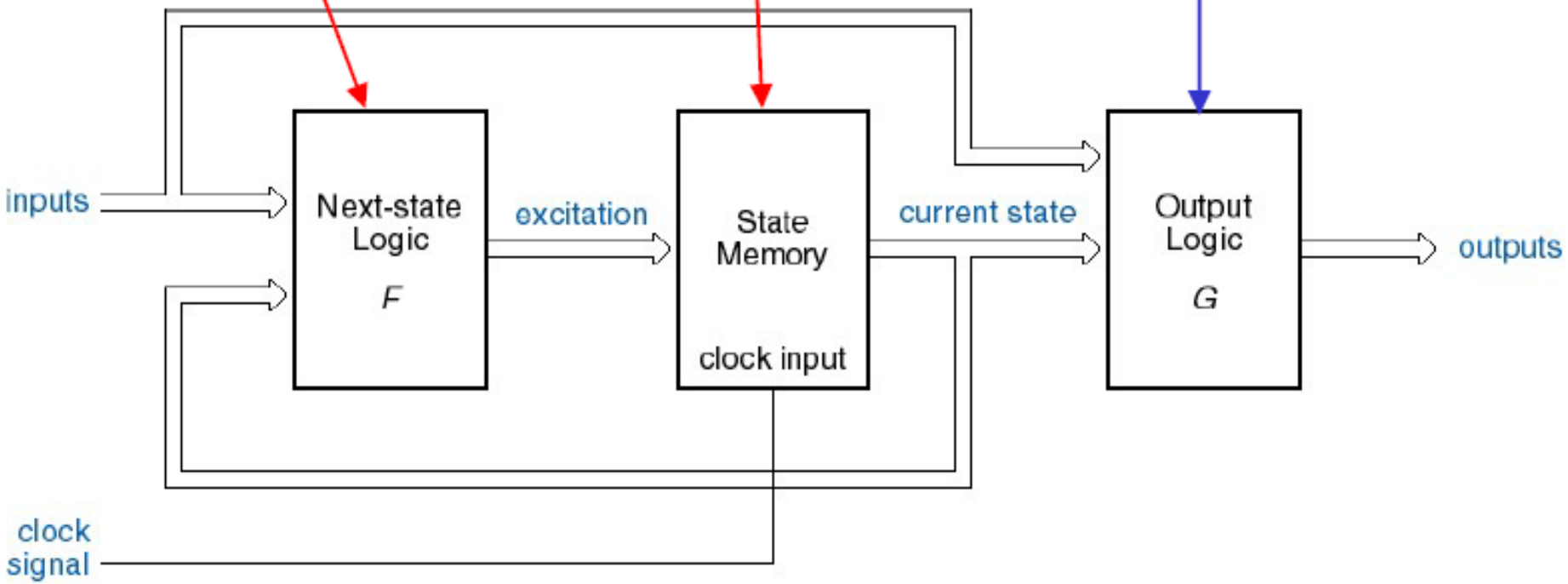
- 三段式（推荐）：使用3个always模块:
  - 第1个always模块采用同步时序描述状态转移；
  - 第2个采用组合逻辑判断状态转移条件、描述状态转移规律；
  - 第3个always模块使用同步时序电路描述每个状态电路对应的输出。
- 两段式FSM描述有一个明显的弱点就是其输出一般使用组合逻辑描述，而组合逻辑易产生毛刺等不稳定因素，并且在FPGA、CPLD等逻辑器件中过多的组合逻辑会影响实现的速率。
- 三段式FSM描述方法使用同步时序逻辑寄存FSM的输出，可以减少毛刺的产生。

三段式FSM描述结构

第2个always，纯组合逻辑，描述触发器的驱动逻辑

第1个always，同步时序，格式化地描述状态转换：
state<=nextstate

第3个always，同步时序，插入寄存器以寄存电路的输出



状态机设计的其他技巧

- FSM的编码：Binary（二进制编码）、gray-code（格雷码）编码使用最少的触发器、较多的组合逻辑；one-hot（独热码）编码反之。小型设计使用gray-code和binary编码更有效，而大型状态机使用onehot更高效， FPGA多使用one-hot编码。
- FSM初始化状态：当芯片加电或者复位后，状态机应该能够自动将所有判断条件复位，并进入初始化状态。一般的方法是采用异步复位信号。
- FSM的默认状态：完整的状态机应该包含一个默认(default)状态，当转移条件不满足，或者状态发生了突变时，要能保证逻辑不会陷入“死循环”。这是对状态机健壮性的一个重要要求，也就是常说的要具备“自恢复”功能。If-else语句要特别注意，尽量使用完备的条件判断语句。在case语句结构中增加default默认状态是推荐的代码风格。

实例1：110序列检测器

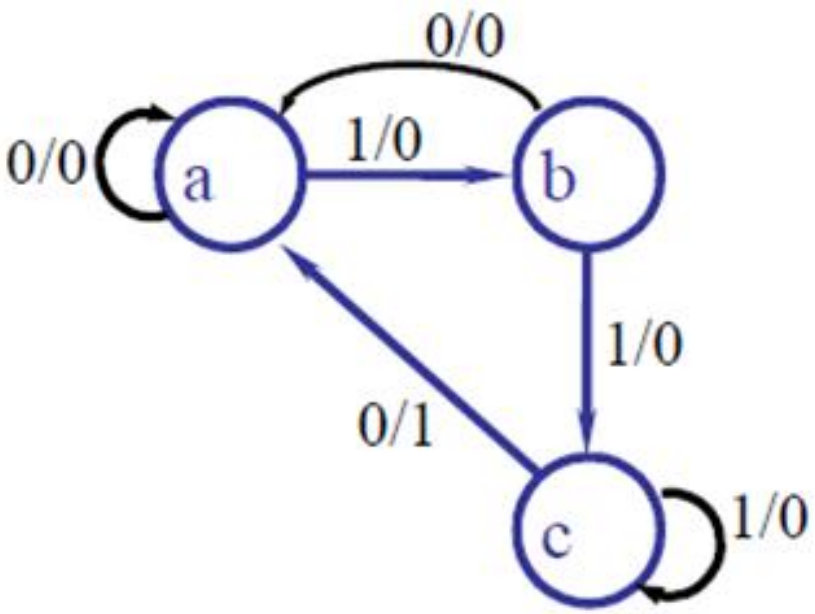
设计一个110序列检测器。当接收到的序列信号中出现110时，输出Y为1，其它输出为0.

解：状态含义

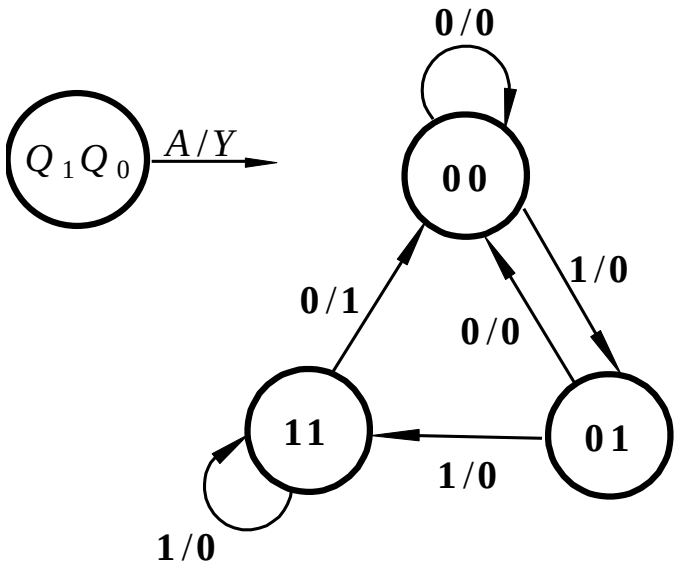
a:未收到有效序列信号时的电路状态，该状态也是初始状态；

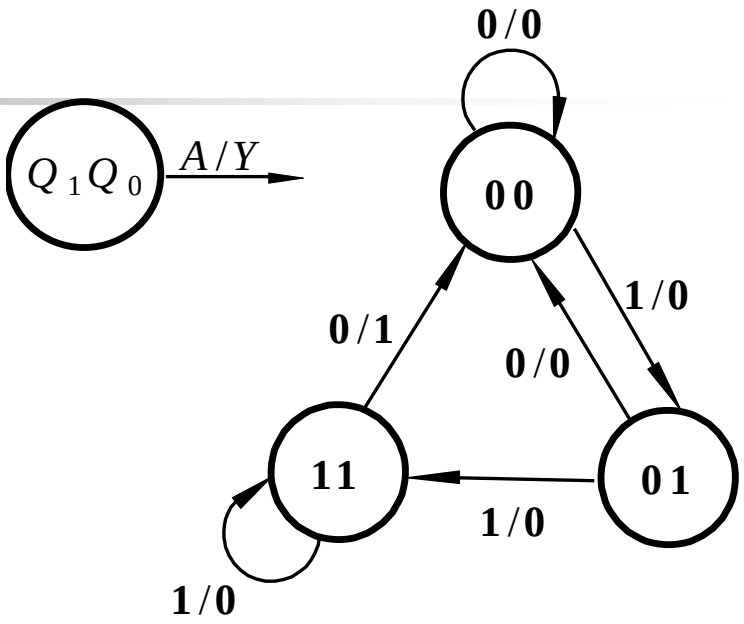
b:收到“1”后的状态；

c:收到“11”后的状态。



```
module Mealy_sequence_detector (A, CP, CR, Y);
input A, CP, CR;
output Y;
reg Y;
reg [1:0] current_state, next_state;
parameter S0=2'b00, S1=2'b01, S2=2'b11;
//第一个always，状态转换
always @( posedge CP or negedge CR)
begin
if (~CR) current_state <=S0; //在CR下降沿设s0为初态
else current_state <= next_state;
end
//第二个always，次态逻辑
always @(current_state or A)
begin
case(current_state)
S0: next_state=(A==1)? S1: S0;
S1: next_state=(A==1)? S2: S0;
S2: next_state=(A==1)? S2: S0;
default: next_state=S0;
endcase
end
```





//output logic

两段式描述：

assign Y=(current_state==S2)&(A==0);
endmodule

三段式描述：第三个always (Y是寄存器型)

always @(posedge CP , negedge CR)

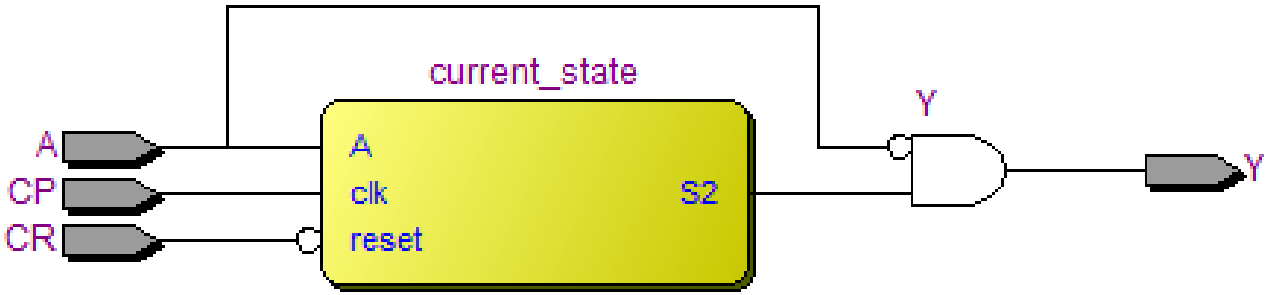
if (~CR) Y<=1’b0;

else Y<=(current_state==S2)&(A==0);

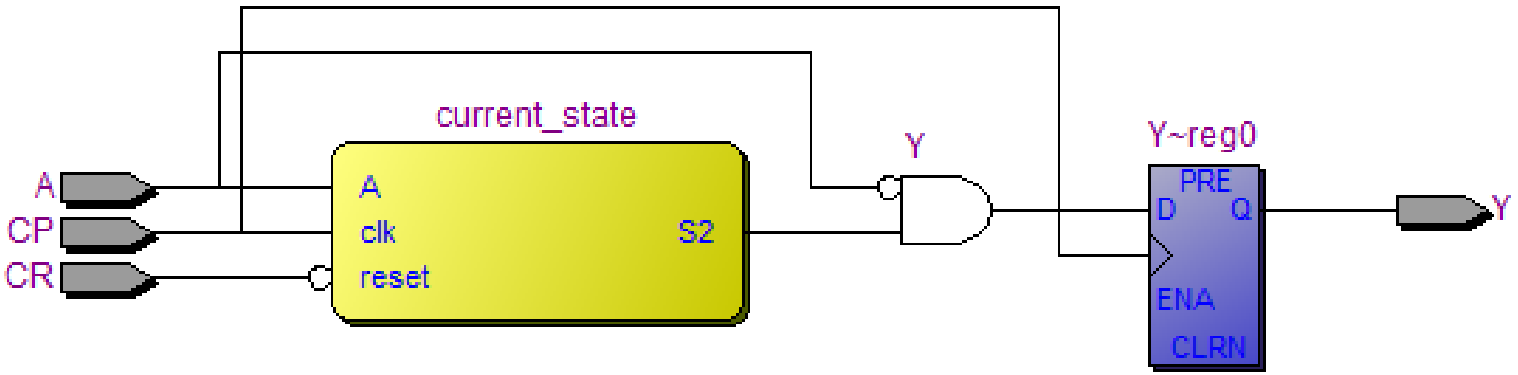
endmodule

110序列检测器综合结果

两段式
综合结果

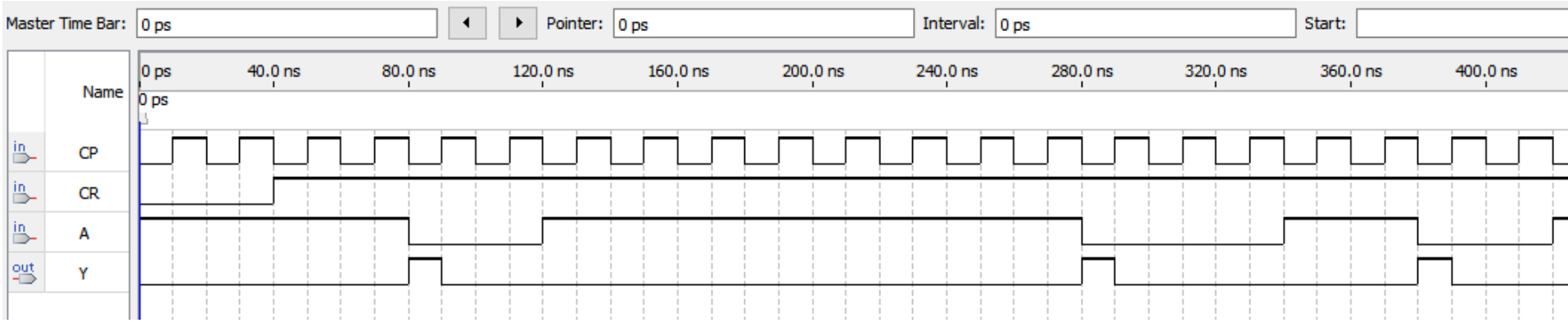


三段式
综合结果

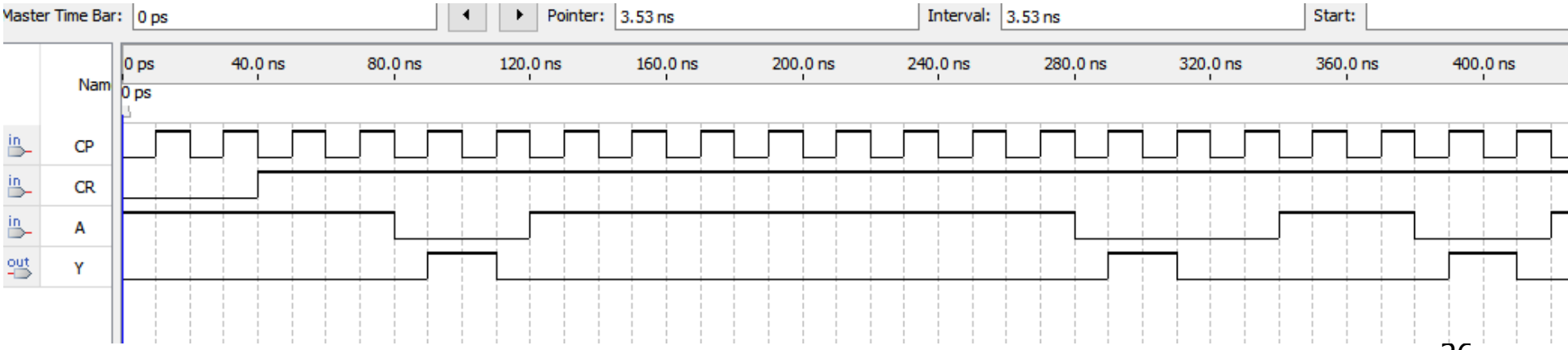


110序列检测器仿真结果（注意输出Y）

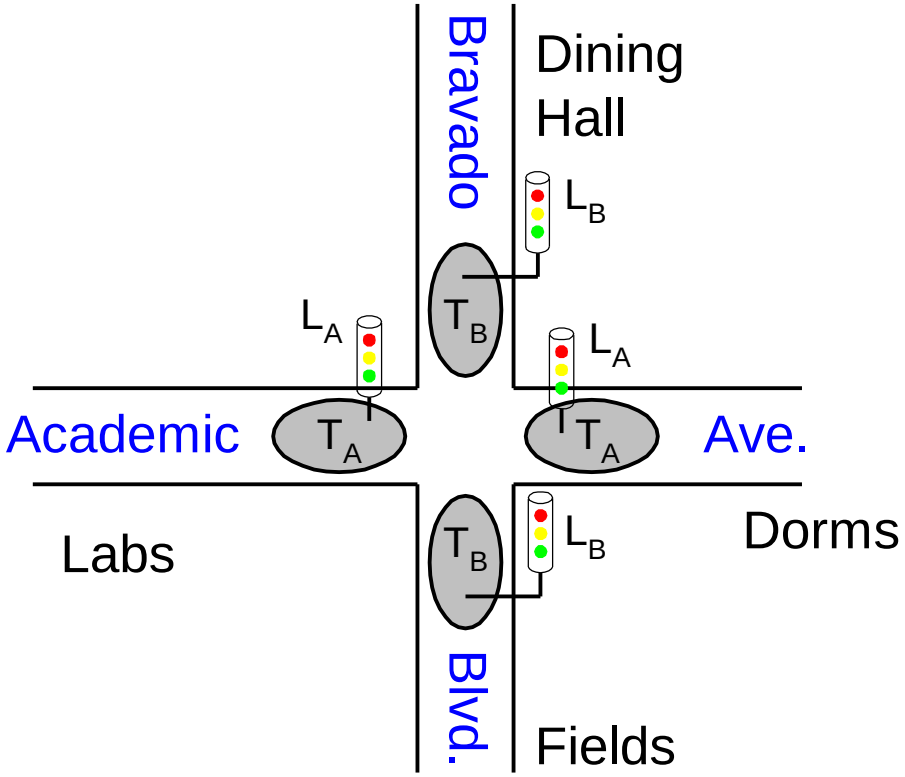
两段式仿真结果

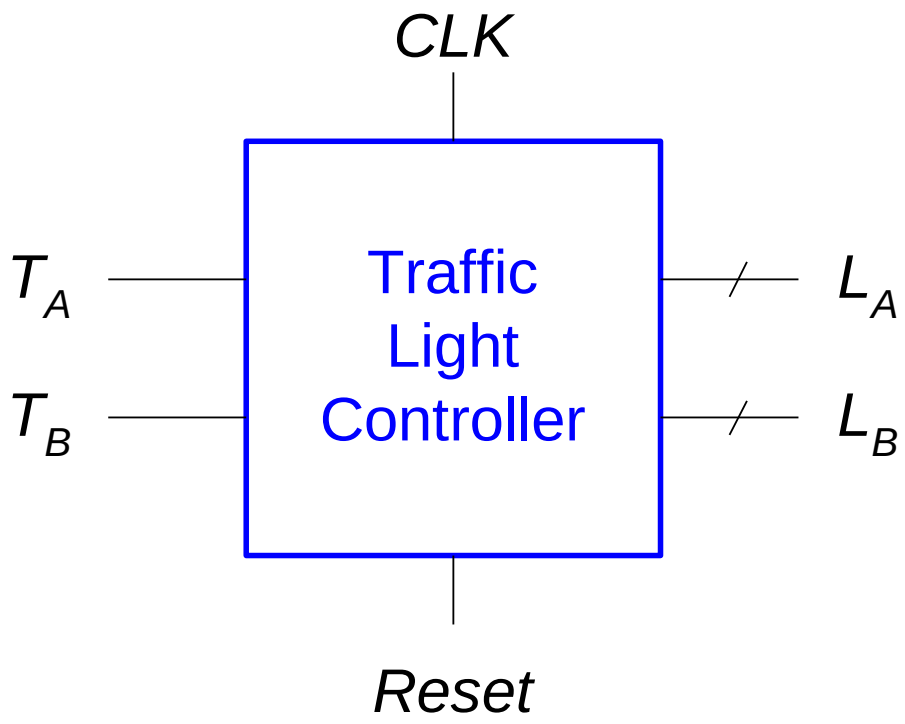


三段式仿真结果

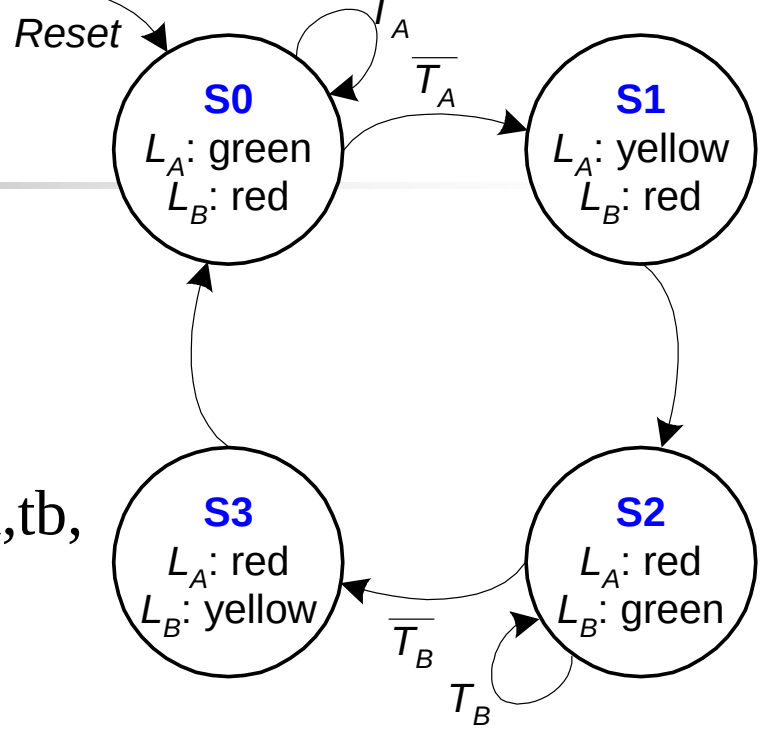


实例2：交通灯控制器

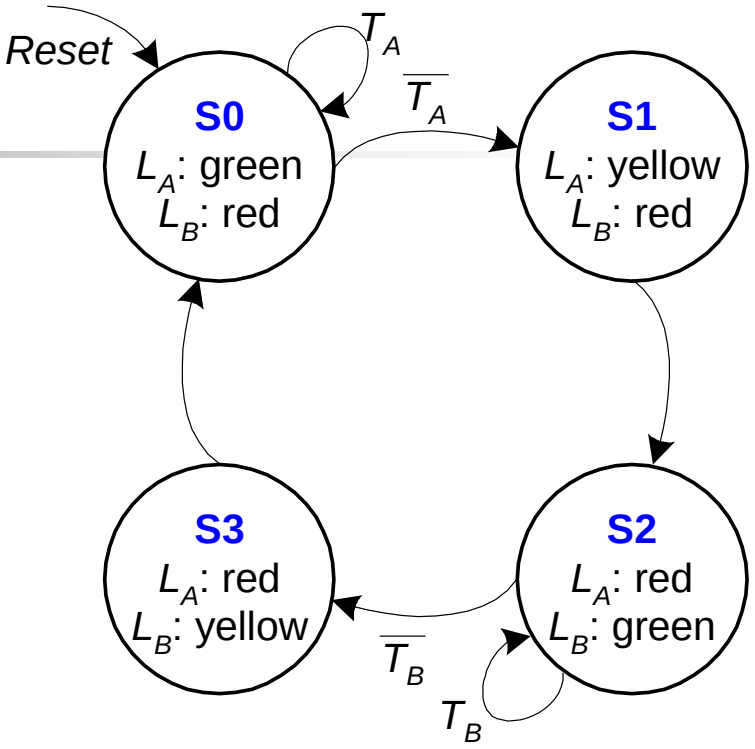

Bravado
Dining Hall
L_B
L_A
T_B
L_A
Academic
T_A
T_A
Ave.
Labs
T_B
L_B
Dorms
Blvd.
Fields


CLK
T_A
T_B
Traffic Light Controller
L_A
L_B
Reset

交通灯控制器代码




```
module traffic_controller(input clk,reset,ta,tb,
output reg[1:0]la,lb);
reg [1:0] state,nextstate;
parameter S0=2'b00;//复位或TA=1，A道绿，B道红
parameter S1=2'b01;//TA=0，A道黄，B道红
parameter S2=2'b10;//TB=1，A道红，B道绿
parameter S3=2'b11;//TB=0，A道红，B道黄
parameter green=2'b00;//绿
parameter yellow=2'b01;//黄
parameter red=2'b10;//红
```



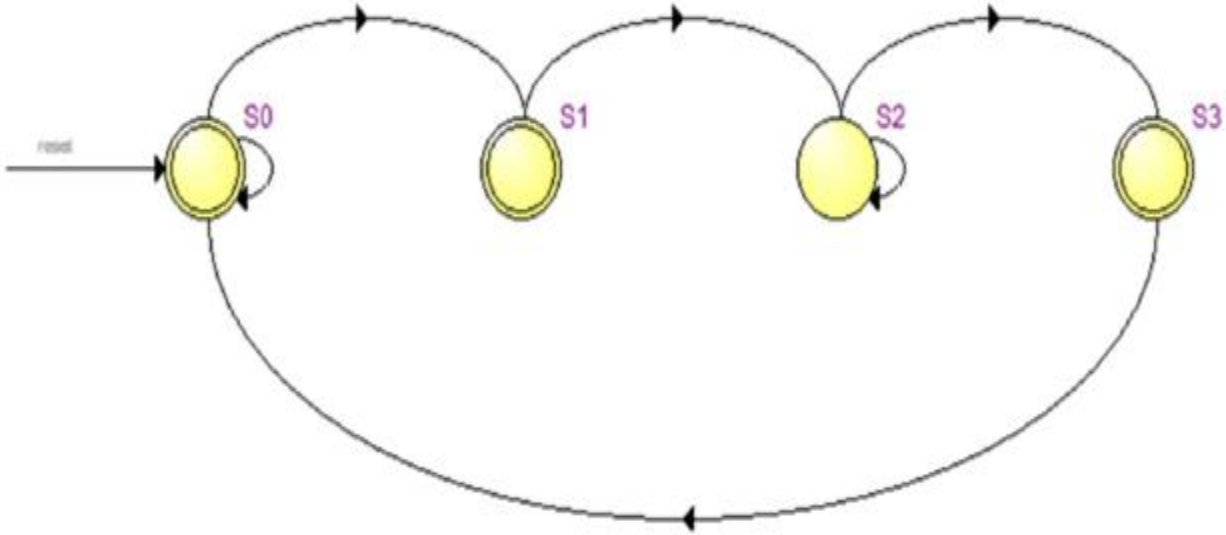

```
//第1段，State Register
always@(posedge clk,posedge reset)
   if(reset)state<=S0;
else state<=nextstate;
```

```
//第2段，Next State Logic
always@(*)
 case (state)
    S0:if (ta) nextstate=So;
         else extstate=S1;
    S1:nextstate=S2;
    S2:if (tb) nextstate=S2;
        else nextstate=S3;
    S3: nextstate=S0;
endcase
```

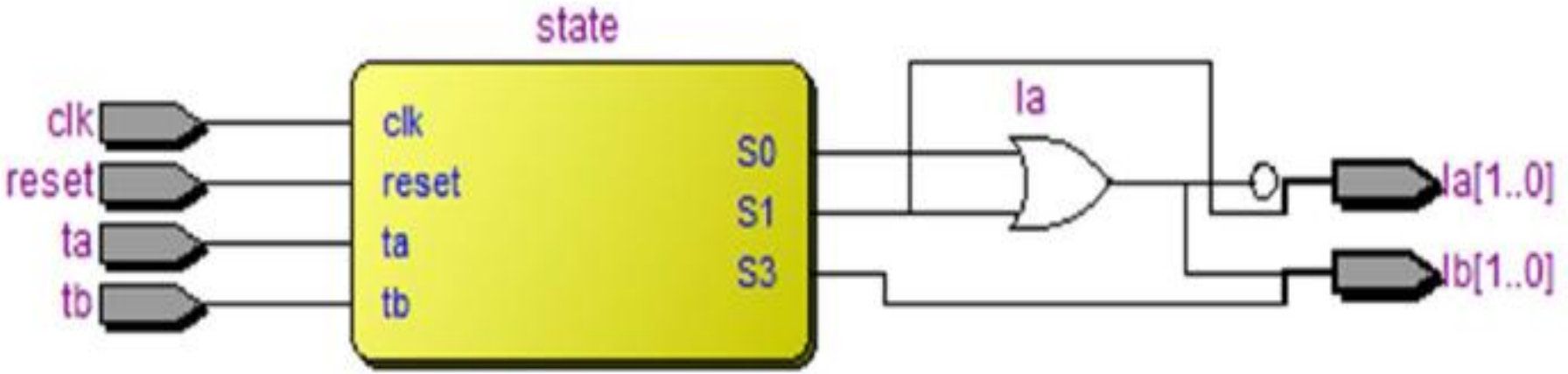
```
//Output  Logic
always@(*)
  case (state)
     S0:{la,lb}={green, red};
     S1:{la,lb}={yellow, red};
     S2:{la,lb}={red, green};
     S3:{la,lb}={red, yellow};
endcase
endmodule
```

交通灯控制器综合结果

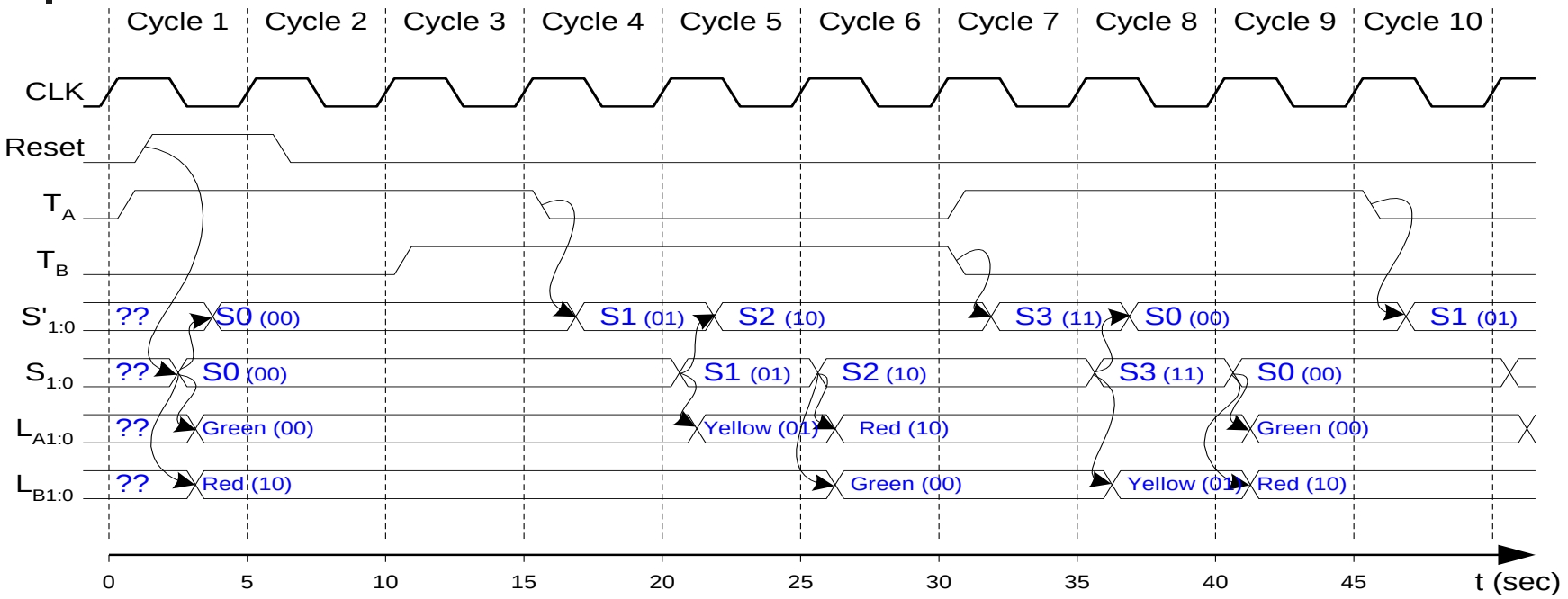
综合出的状态机

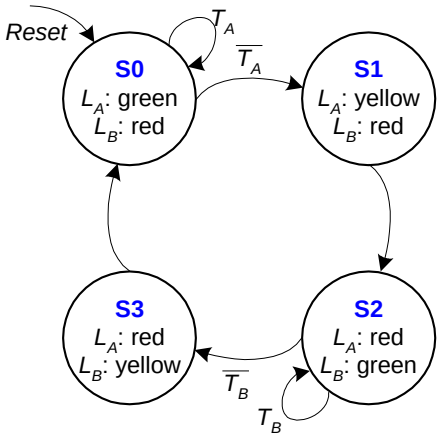


两段式综合结果



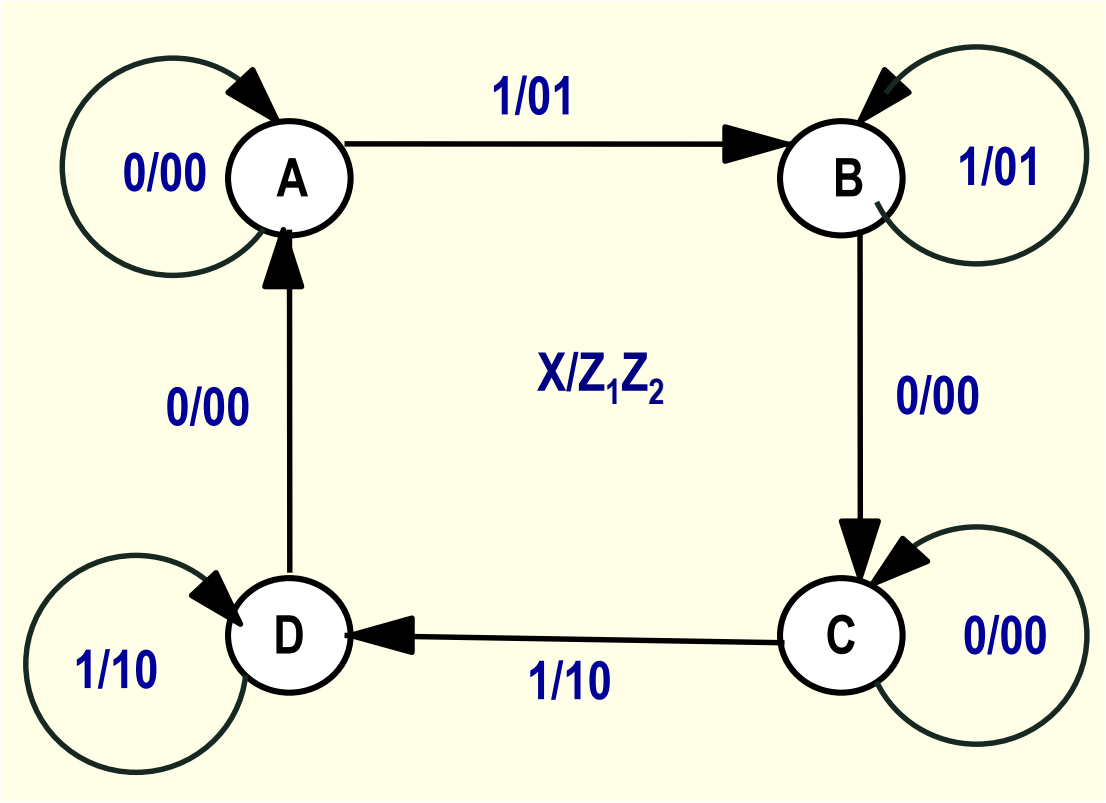
交通灯控制器仿真验证





FPGA芯片有50MHz的时钟信号输出，如何得到交通灯控制器中5s周期的时钟？

实例3：机器人控制器



两段式：

```
module  robotcontroller (X, CP, Z1, Z2);
    input X, CP;
    output Z1,Z2;
    reg Z1,Z2;
    reg [1:0] current_state, next_state;
    parameter  A=2'b00, B=2'b01,
C=2'b11, D=2'b10;

//第一段，状态转换
always @( posedge CP )
  current_state <= next_state;
```

```
//第二段，次态、输出逻辑
always @(current_state or X)
   case(current_state)
     A: if (X) begin next_state=B; {Z1,Z2}=2'b01; end
        else begin next_state=A; {Z1,Z2}=2'b00; end
     B: if (X) begin next_state=B; {Z1,Z2}=2'b01; end
        else begin next_state=C; {Z1,Z2}=2'b00; end
     C: if (X) begin next_state=D; {Z1,Z2}=2'b10; end
        else begin next_state=C; {Z1,Z2}=2'b00; end
     D: if (X) begin next_state=D; {Z1,Z2}=2'b10; end
              else begin next_state=A; {Z1,Z2}=2'b00; end
      default: begin next_state=A; {Z1,Z2}=2'b00; end
   endcase
endmodule
```

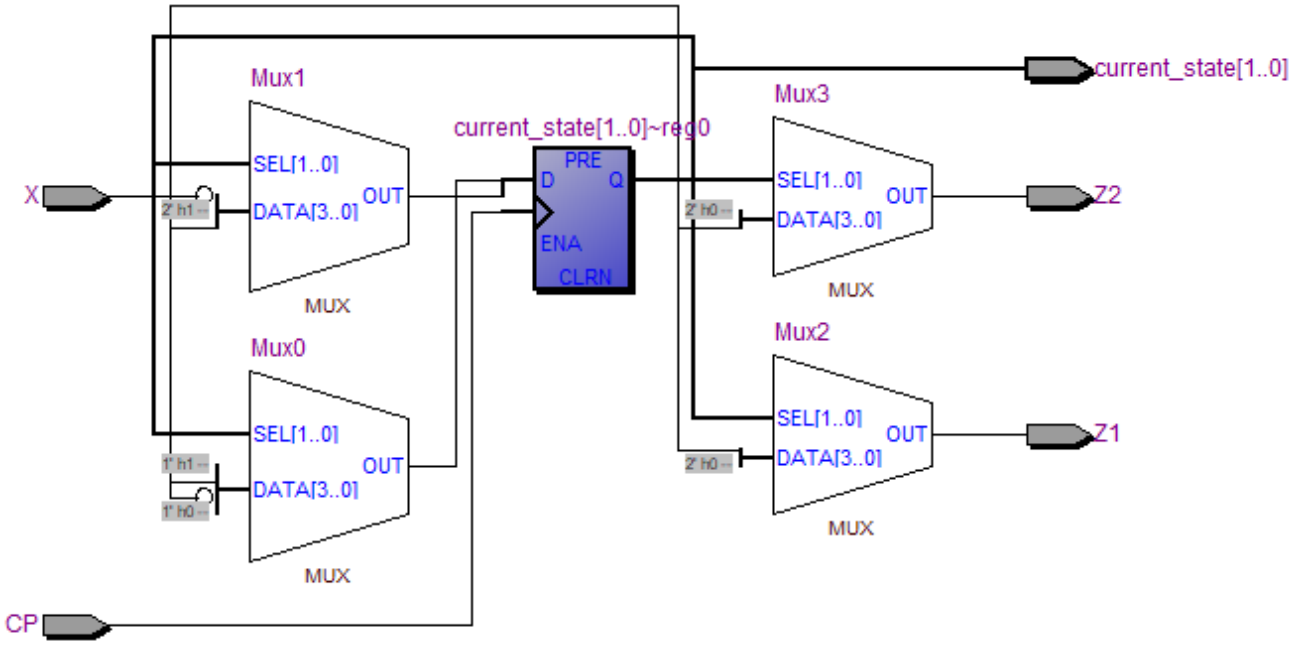
三段式：

```
module  robotcontroller_3seg (X, CP, Z1, Z2,current_state);
    input X, CP;
    output Z1,Z2;
    output reg[1:0]current_state;
    reg Z1,Z2;
    reg [1:0]  next_state;
    parameter  A=2'b00, B=2'b01, C=2'b11, D=2'b10;
//第一段，状态转换
always @( posede CP )
   current_state <= next_state;

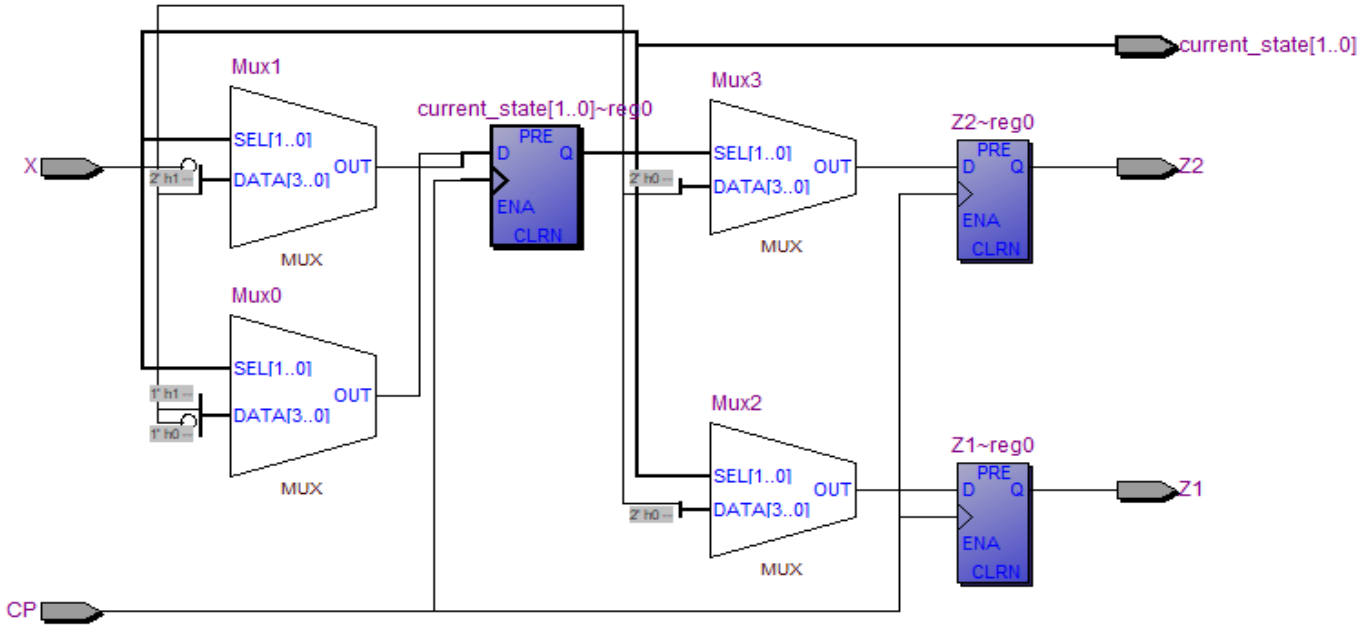
//第二段，次态逻辑
always @(current_state or X)
   case(current_state)
     A: if (X)  next_state=B;  else next_state=A;
     B: if (X)  next_state=B;  else  next_state=C;
     C: if (X)  next_state=D;  else next_state=C;
     D: if (X)  next_state=D;  else next_state=A;
      default:  next_state=A;
   endcase

//第三段，输出逻辑
always @(posedge CP)
   case(current_state)
     A: if (X) {Z1,Z2}<=2'b01;    else {Z1,Z2}<=2'b00;
     B: if (X) {Z1,Z2}<=2'b01;    else {Z1,Z2}<=2'b00;
     C: if (X) {Z1,Z2}<=2'b10;    else {Z1,Z2}<=2'b00;
     D: if (X) {Z1,Z2}<=2'b10;    else {Z1,Z2}<=2'b00;
      default:{Z1,Z2}<=2'b00;
   endcase
endmodule
```

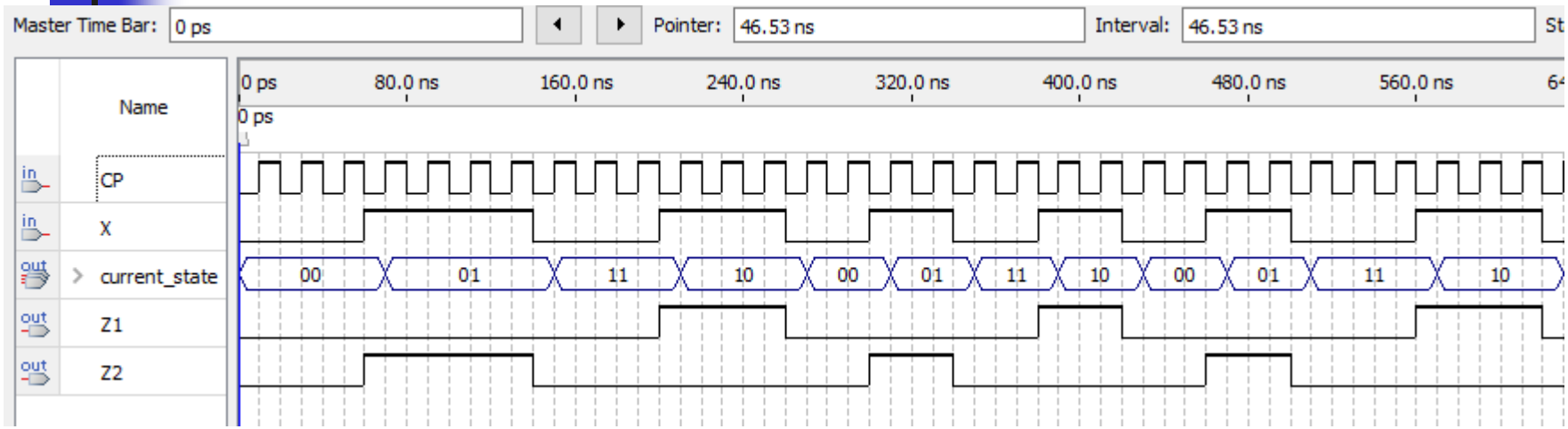
两段式综合结果



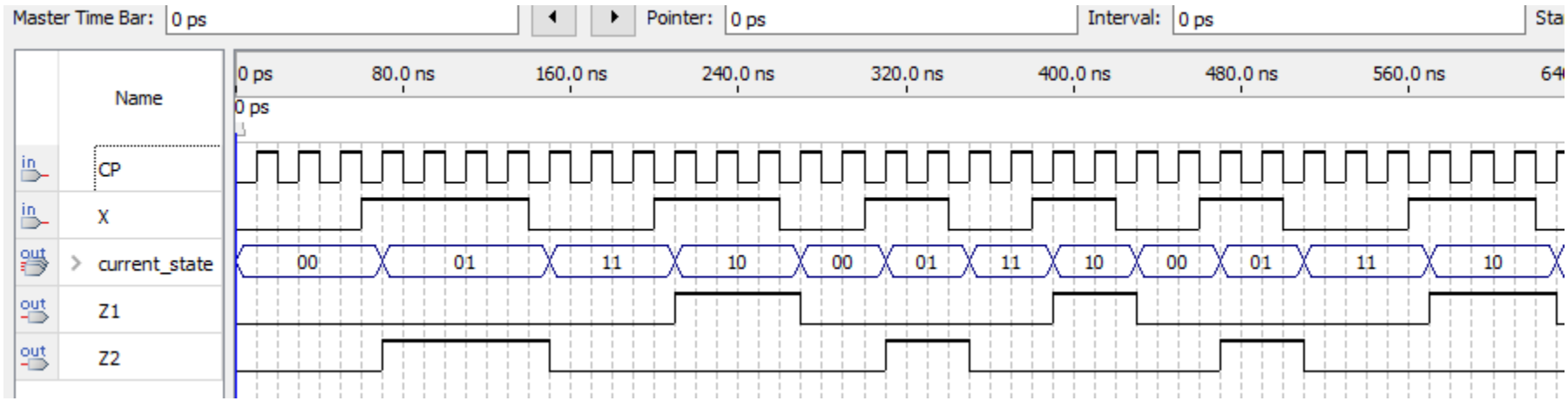
三段式综合结果



两段式仿真结果：

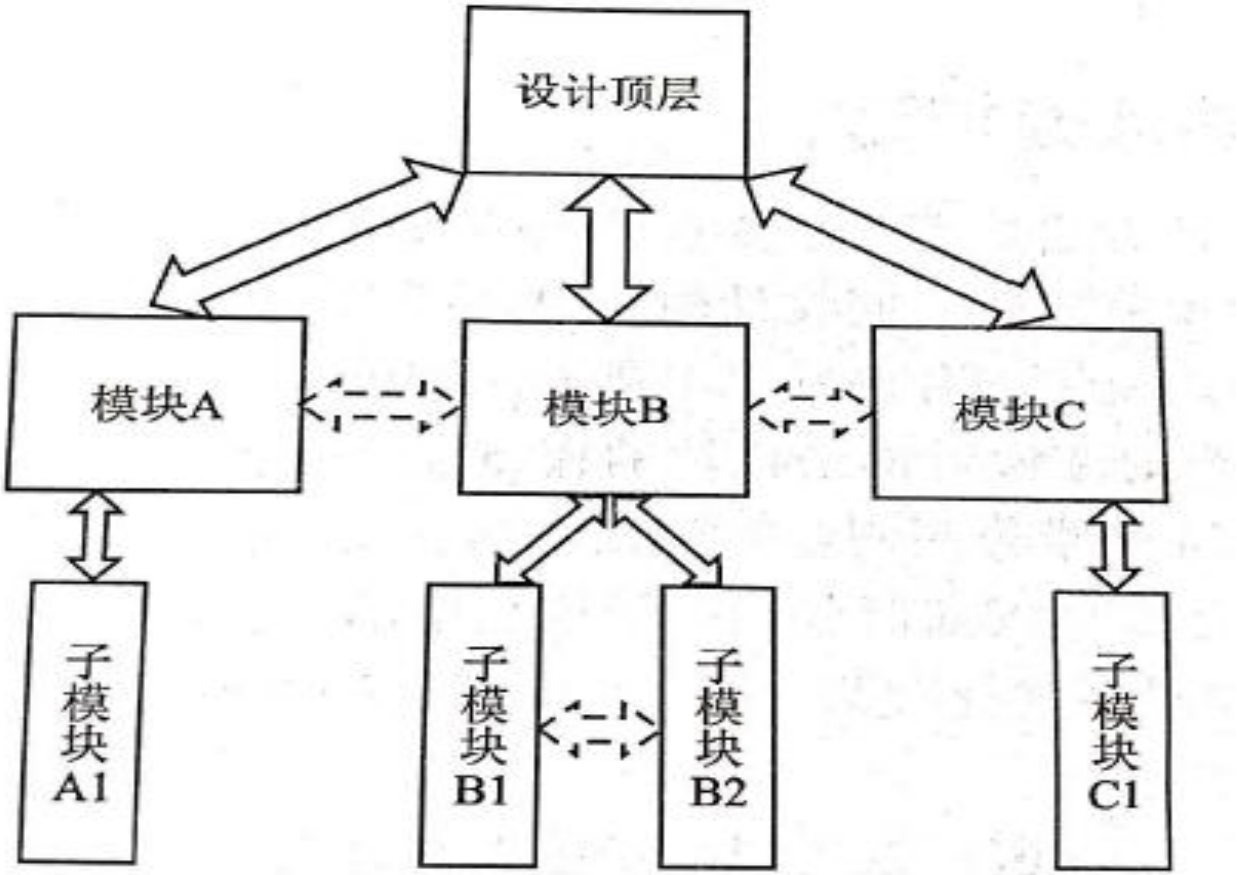


三段式仿真结果：



结构层次化

编码结构层次化(Hierarchical Coding)是模块化设计思想的一种体现。目前大型设计中必须采用结构层次化编码风格，以提高代码的可读性，易于模块划分，易于分工协作，易于设计仿真测试激励。



实例4：步进电机控制器

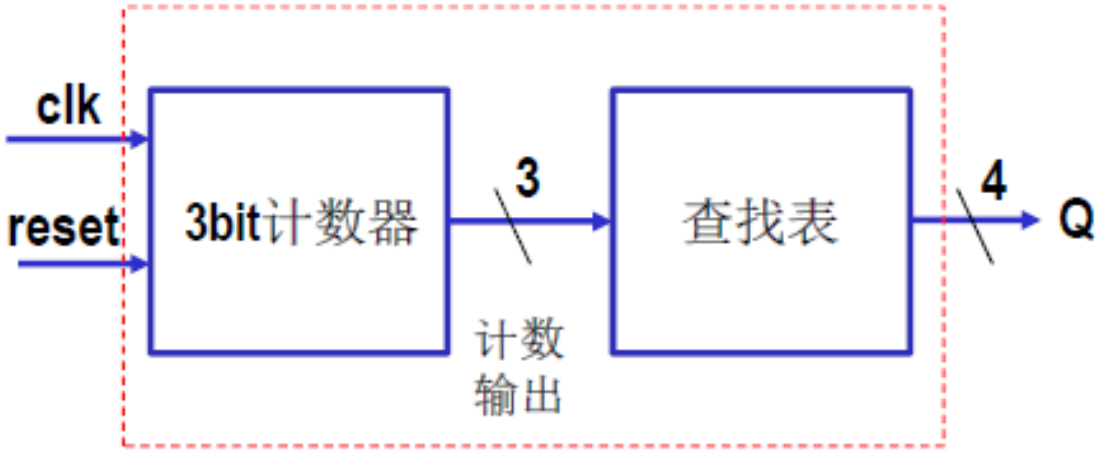
用verilog描述步进电机控制器，状态表如下：

步序	Q3Q2Q1Q0
0	1010
1	1000
2	1001
3	0001
4	0101
5	0100
6	0110
7	0010

模块化：

0~7计数 ，3位计数器

查找表：完成Q[3]~Q[0]四相输出

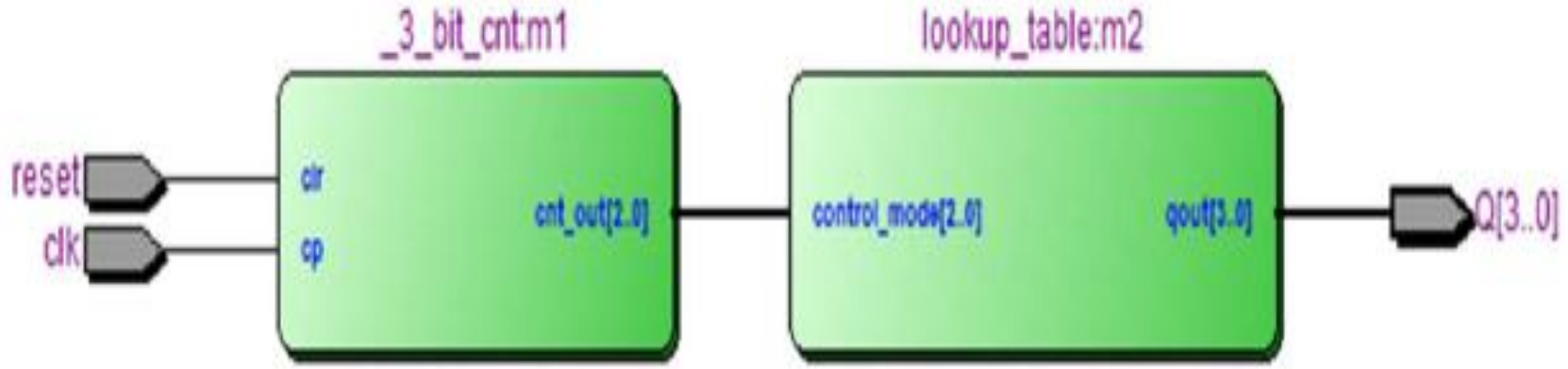


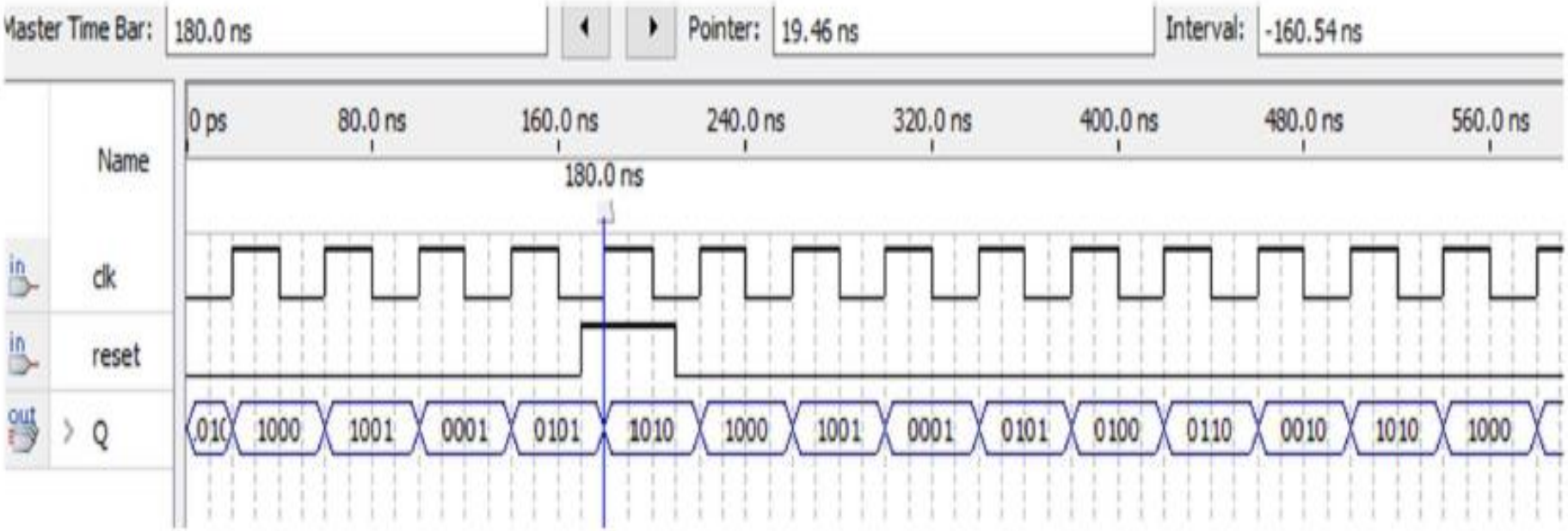
步进电机控制器代码

```
//顶层模块，只有端口定义、变量说明和模块调用
module _4_phase_mt_ctl (clk,reset,Q);
input clk,reset;
output [3:0]Q;
//计数器输出
wire [2:0]motor_cnt;
//调用3bit计数器
_3_bit_cnt m1(.cp(clk),.clr(reset),.cnt_out(motor_cnt));
//调用查找表译码输出步进控制
lookup_table m2(.qout(Q),.control_mode(motor_cnt));
endmodule
```

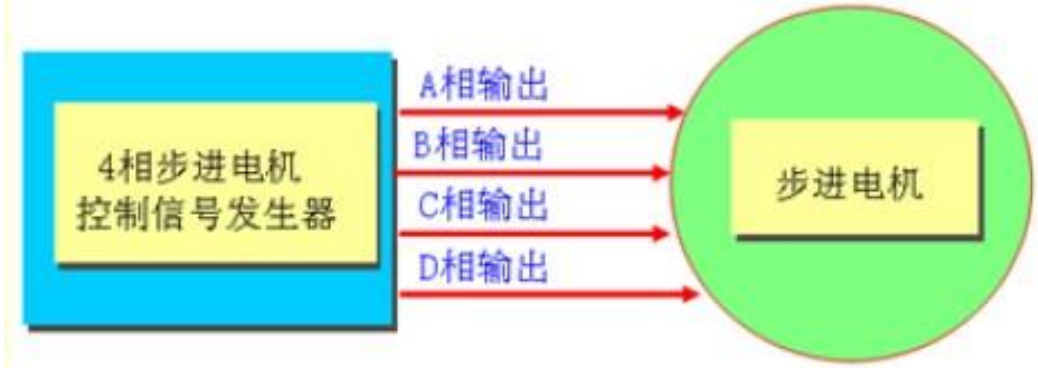
```
//3bit计数器
module _3_bit_cnt(cp,clr,cnt_out);
input cp,clr;
output reg[2:0]cnt_out;
always @(posedge cp)
if (clr) cnt_out <= 3'b000;
else cnt_out <= cnt_out + 1'b1;
endmodule
```

```
//查找表译码，步进控制输出
module lookup_table
(qout,control_mode);
input [2:0]control_mode;
output reg[3:0] qout;
always @(control_mode)
case (control_mode)
3'b000:qout = 4'b1010;
3'b001:qout = 4'b1000;
3'b010:qout = 4'b1001;
3'b011:qout = 4'b0001;
3'b100:qout = 4'b0101;
3'b101:qout = 4'b0100;
3'b110:qout = 4'b0110;
3'b111:qout = 4'b0010;
default qout = 4'b1010;
endcase
endmodule
```


_3_bit_cnt:m1
lookup_table:m2
reset
clk
clr
cp
cnt_out[2..0]
control_mode[2..0]
qout[3..0]
Q[3..0]


Master Time Bar: 180.0 ns
Pointer: 19.46 ns
Interval: -160.54 ns
Name
0 ps
80.0 ns
160.0 ns
240.0 ns
320.0 ns
400.0 ns
480.0 ns
560.0 ns
180.0 ns
clk
reset
Q
010
1000
1001
0001
0101
1010
1000
1001
0001
0101
0100
0110
0010
1010
1000

实例5：多相电机控制



CLK	RS	EN	M	FH	操作
↑	1	X	X	X	置数
↑	0	1	0	1	全步逆
↑	0	1	1	1	全部顺
↑	0	1	0	0	半步逆
↑	0	1	1	0	半步顺
↑	0	0	X	X	保持

全步

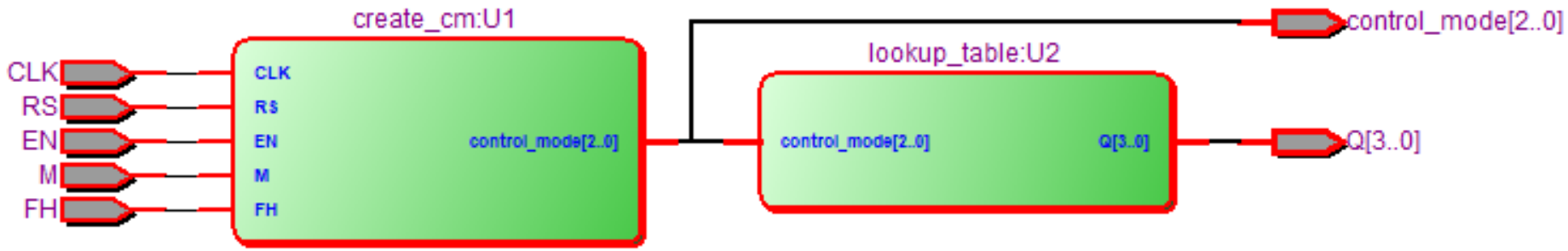
顺时针↓	步序	$Q_3Q_2Q_1Q_0$	↑逆时针
	1	1010	
	2	1000	
	3	1001	
	4	0001	
	5	0101	
	6	0100	
	7	0110	
	8	0010	

半步

顺时针↓	步序	$Q_3Q_2Q_1Q_0$	↑逆时针
	1	1010	
	2	1001	
	3	0101	
	4	0110	

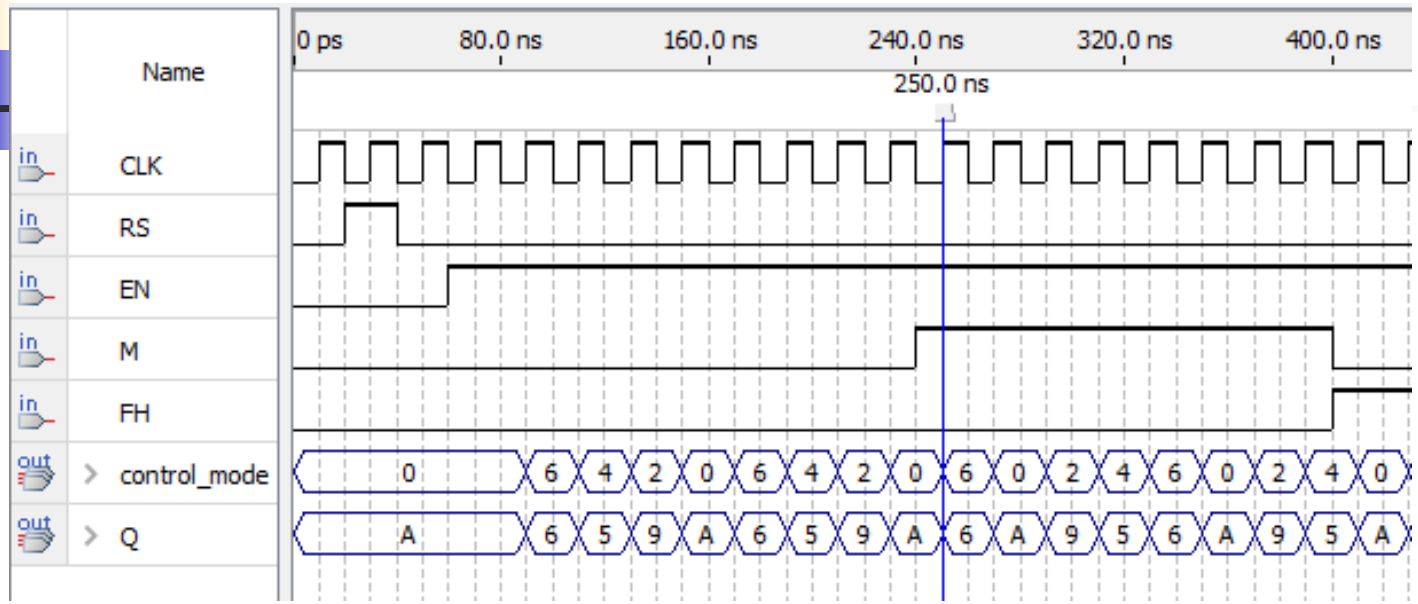
RS：复位，高有效
EN：使能，高有效
M：0/1-逆时针/顺时针
FH：0/1-半步/全步

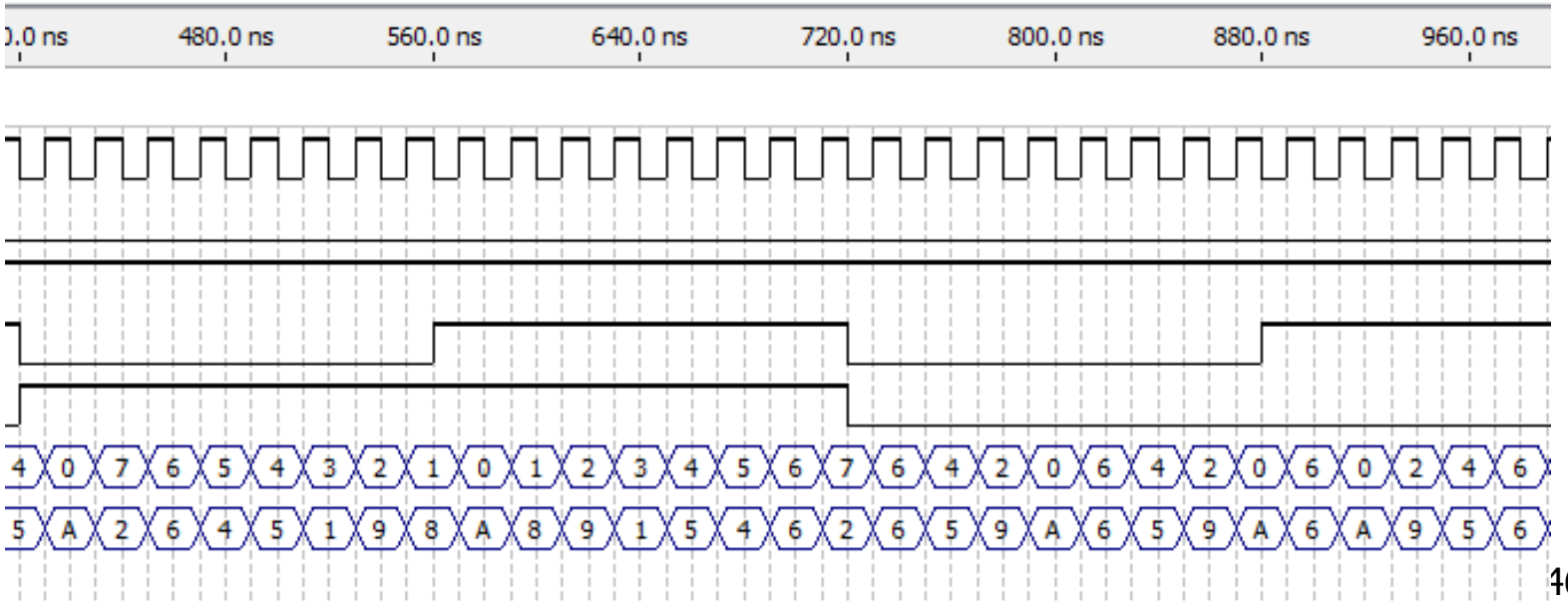
```
module motorcontrol(CLK,RS,EN,M,FH,Q,control_mode); //主模块
input CLK;        //时钟
input RS,EN,M,FH; //置数、使能、模式
output [3:0]Q;    //电机控制
output [2:0]control_mode;
create_pt U1(CLK,RS,EN,M,FH,control_mode);  //步进产生模块
lookup_table U2(Q,control_mode);          //查找表模块
endmodule
```



```
module create_cm(CLK,RS,EN,M,FH,control_mode);  // 控制模式产生模块
input CLK;
input RS,EN,M,FH;  //RS：复位；EN：使能；M、FH：模式控制
output reg[2:0]control_mode;  //控制模式寄存器
reg [2:0]half_cm,full_cm;  // half_cm:半步，full_cm:全步
always @(posedge CLK)  //根据M和FH产生控制模式
  if (RS)
     begin  full_cm<=3'b000;  half_cm<=3'b000; end  //RS有效，复位
  else if (~EN)  begin  full_cm<=full_cm;  half_cm<=half_cm;  end  //EN无效，保持
  else                //RS无效，EN有效，产生控制模式
    case({M,FH}) //根据M和FH产生全步逆、全步顺、半步逆、半步顺控制模式
       2'b01:full_cm<=full_cm-2'b01;
       2'b00:half_cm<=half_cm-2'b10;
       2'b10:half_cm<=half_cm+2'b10;
       2'b11:full_cm<=full_cm+2'b01;
       default  begin  half_cm<=3'b000; full_cm<=3'b000;  end
    endcase
  always @(posedge CLK)  //将产生的控制模式赋值给control_mode寄存器
    if (FH) control_mode<=full_cm;
            else control_mode<=half_cm;
endmodule
```

```
module lookup_table(Q,control_mode);  //查找表模块
input [2:0] control_mode;
output reg[3:0] Q;
always @(control_mode)
  case (control_mode)
    3'b000:Q=4'b1010;
    3'b001:Q=4'b1000;
    3'b010:Q=4'b1001;
    3'b011:Q=4'b0001;
    3'b100:Q=4'b0101;
    3'b101:Q=4'b0100;
    3'b110:Q=4'b0110;
    3'b111:Q=4'b0010;
    default  Q=4'b1010;
  endcase
endmodule
```


Name
0 ps
80.0 ns
160.0 ns
240.0 ns
320.0 ns
400.0 ns
250.0 ns
CLK
RS
EN
M
FH
control_mode
Q
0 6 4 2 0 6 4 2 0 6 0 2 4 6 0 2 4 0
A 6 5 9 A 6 5 9 A 6 A 9 5 6 A 9 5 A


.0 ns
480.0 ns
560.0 ns
640.0 ns
720.0 ns
800.0 ns
880.0 ns
960.0 ns
4 0 7 6 5 4 3 2 1 0 1 2 3 4 5 6 7 6 4 2 0 6 4 2 0 6 0 2 4 6
5 A 2 6 4 5 1 9 8 A 8 9 1 5 4 6 2 6 5 9 A 6 5 9 A 6 A 9 5 6

实例6：三角波发生器

```
//加减计数模块
module updwncnt(freq,q);
input freq;
output reg[3:0]q;
reg updwn;

always @(posedge freq)
 if (~updwn)
   begin
   q=q+1'b1;
   if (q==4'b1111)
       updwn=1'b1;
   end
 else
   begin
   q=q-1'b1;
   if (q==4'b0000)
      updwn=1'b0;
   end

endmodule
```

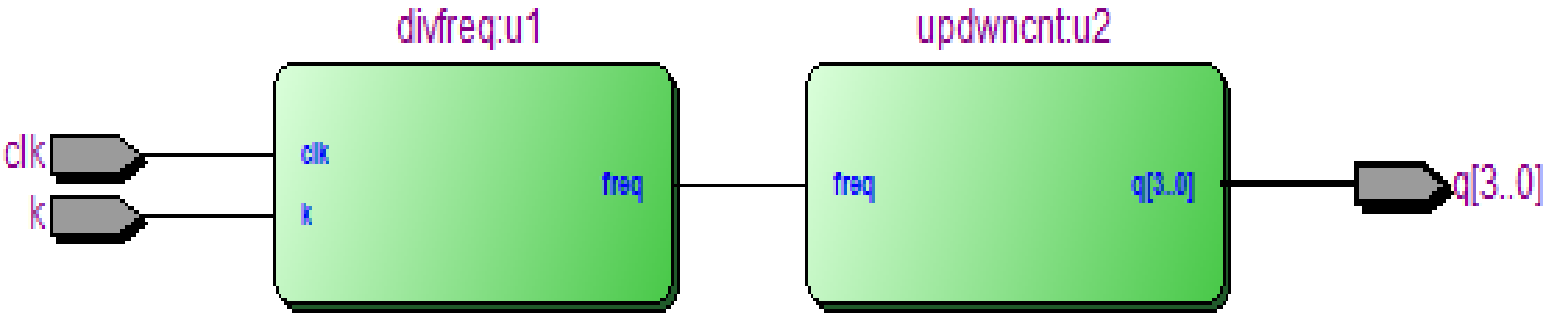
```
//分频模块
module divfreq(clk,k,freq);
input clk, k;
output freq;
reg temp_freq;
reg [3:0]cnt;

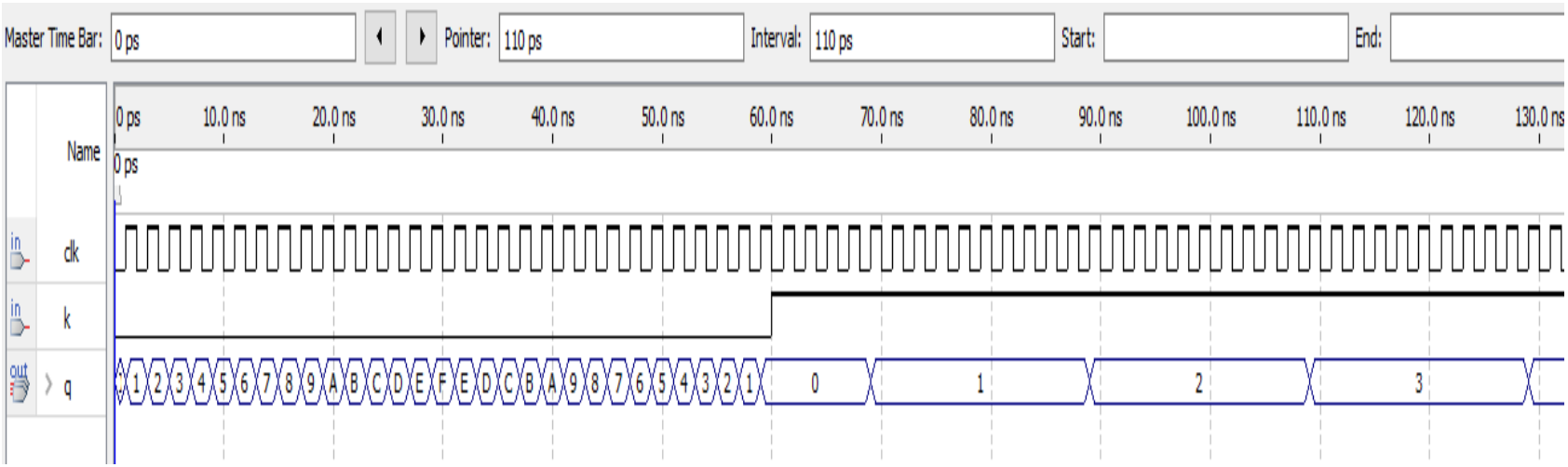
parameter par=4'd4;

always @(posedge clk )
    if (cnt>=par)
      begin
      cnt<=4'd0;
      temp_freq<=~temp_freq;
      end
    else
      cnt<=cnt+1'b1;

 assign freq=(k?temp_freq:clk);
 endmodule
```

```
//主模块
module tri_wave(clk,k,q);
input clk;
input k;
output [3:0]q;
wire freq;
divfreq u1(clk,k,freq);
updwncnt u2(freq,q);
endmodule
```


divfreq:u1
updwncnt:u2
clk
k
clk
k
freq
freq
q[3..0]
q[3..0]


Master Time Bar: 0 ps
Pointer: 110 ps
Interval: 110 ps
Start:
End:
Name
0 ps
10.0 ns
20.0 ns
30.0 ns
40.0 ns
50.0 ns
60.0 ns
70.0 ns
80.0 ns
90.0 ns
100.0 ns
110.0 ns
120.0 ns
130.0 ns
clk
k
q
0
1
2
3